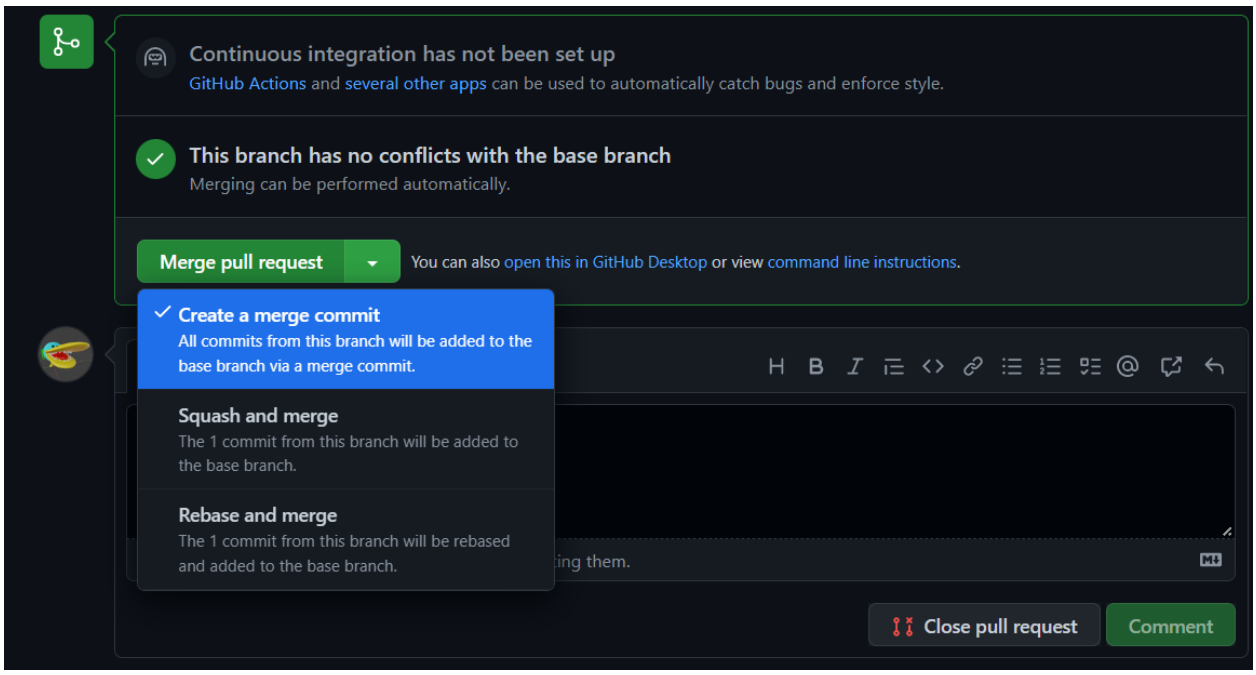
Описание популярных методов объединения веток в git

[Метод merge](#)

[Метод rebase](#)

[Метод squash](#)

Три наиболее популярных методов слияния веток git это:
merge, rebase и squash. Данные опции, например, предлагаются в GitHub при выборе метода, по которому будет выполнен Merge pull request:



У каждого из этих методов есть свои недостатки и преимущества. Выбор того какой метод использовать зависит от предпочтений команды. В данном документе кратко описывается каждый метод с перечислением преимуществ и недостатков каждого

Объединений веток методом merge

Под merge методом подразумевается выполнение команды `git merge <feature>`, когда HEAD находится на ветке, в которую вносятся изменения из ветки `<feature>`. Данная команда создаёт на текущей ветке новый merge-коммит, который имеет два parent-коммита: последний коммит текущей ветки и последний коммит слитой ветки. Если конфликт веток не был разрешён в самой ветке `<feature>`, то этот merge-коммит может также содержать разрешение конфликтов.

```
        A---B---C        feature - ветка, которую слили в master
       /         \
D---E---F---G---M      master - ветка, в которую был произведён merge
                       M - merge-коммит
```

Преймущества данного подхода:

- Последствия слияния можно отменить в любой момент, например, сделав revert merge-коммита;

- Понятна история разработки;

- Сохраняется весь когда-либо написанный код, что может быть полезно, например при исправлении багов или если позже в ходе разработки выяснится, что конфликт был разрешён неверно;

Недостатки данного подхода:

- Так как у merge-коммита два parent-коммита, то при выполнении revert merge-коммита, нужно выяснять и явно указывать какой коммит считать предыдущим;

- По ходу разработки дерево коммитов становится трудным для анализа, становится сложно найти нужный коммит;

- Из-за merge-коммитов может усложниться review, так как они содержат информацию об исправлениях или разрешении конфликтов, а не информацию о законченной реализации определённого функционала;

- Сложнее использовать команду git bisect для поиска коммита в котором появился баг;

Объединений веток методом rebase

Под rebase методом подразумевается выполнение команды `git rebase <newbase>`, когда HEAD находится на ветке, с которой переносятся изменения на ветку `<newbase>`. Данная команда перезаписывает все коммиты текущей ветки.

```
      A---B---C           feature - ветка, из которой берутся изменения
     /
D---E---F---G             master - ветка, в которую переносятся изменения

                                  ↓

              A'--B'--C' feature – новая ветка, в которой разрешены конфликты с master
             /
D---E---F---G             master
```

Преймущества данного подхода:

- Дерево коммитов при таком подходе остаётся линейным и простым;

- Упрощается review;

- Легко использовать команду git bisect для поиска коммита в котором появился баг;

Недостатки данного подхода:

- В ходе rebase теряется оригинальный код и коммиты;

- Процесс разрешения конфликта немного сложнее чем при merge подходе, так как нужно разрешать конфликты для отдельных коммитов, а не сразу для всей ветки. То есть может потребоваться несколько раз разрешить фактически один и тот же конфликт;

- Если текущая ветка уже опубликована в remote-репозиторий, то может понадобится force push;

Объединений веток методом squash

Под squash методом подразумевается выполнение команды `git merge –squash <feature>`, когда HEAD находится на ветке, в которую вносятся изменения из ветки `<feature>`. Данная команда создаёт на текущей ветке новый squash-коммит, который включает в себя все изменения из коммитов ветки `<feature>`. Если конфликт веток не был разрешён в самой ветке `<feature>`, то этот squash-коммит может также содержать разрешение конфликтов.

```
        A---B---C                    feature - ветка, из которой берутся изменения
       /
D---E---F---G---ABC'                 master - ветка, в которую переносятся изменения
                                     ABC' - squash-коммит
```

Преймущества данного подхода:

- Последствия слияния можно легко отменить в любой момент, например, сделав revert squash-коммита;

- Если не удалять слитую ветку, то сохраняется весь когда-либо написанный код, что может быть полезно, например при исправлении багов или если позже в ходе разработки выяснится, что конфликт был разрешён неверно;

- Главные ветки при таком подходе остаются линейными и простыми;

- Можно использовать команду git bisect для поиска коммита в котором появился баг;

Недостатки данного подхода:

- Неудобно использовать, если над сливаемой веткой нужно продолжить работать. При последующем слитии нужно будет повторно разрешать те же конфликты;

- Если оставлять исходную ветку `<feature>` и разрешать конфликт путём предварительного слития в ветку `<feature>`, то история git, может стать сложной для анализа;

- Если удалять исходную ветку `<feature>` и разрешать конфликт в самом squash-коммите, то: может усложниться review; теряется оригинальный код; коммиты и история git становится менее информативной;